# SIMATS ENGINEERING

## ASSESSMENT TOOL 1: Analytical Problem Solving

**COURSE NAME:** Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1518

---

**COURSE OUTCOME COVERED**

**CO2:** *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 50**

Question Count: 5

---

**Code of Conduct**

I DILLILOKESH D (192311097) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Dillilokesh D

**Assessment Tasks**

**Analytical Problem Solving**

1. Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.
2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.
3. Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.
4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.
5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

**Analytical Problem Solving Rubric**

| Criteria | Weightage | Level 4 - Exemplary | Level 3 – Proficient | Level 2 - Developing | Level 1 - Beginning |
|---|---|---|---|---|---|
| Problem Interpretation | 20% | Interprets all technical requirements accurately | Interprets most requirements correctly | Partial interpretation | Misinterprets the problem |
| Analytical Reasoning | 25% | Strong reasoning with clear cause-effect analysis | Good reasoning with minor gaps | Basic analysis only | Weak or no analysis |
| Method Selection | 20% | Chooses highly appropriate virtualization and security concepts | Chooses mostly suitable concepts | Partially suitable concepts | Inappropriate approach |
| Solution Quality | 20% | Provides feasible and technically sound solution | Reasonable solution with minor issues | Basic solution with limitations | Unclear or invalid solution |
| Presentation of Analysis | 15% | Well-structured and technically precise | Generally clear | Somewhat unclear | Difficult to follow |

**Total Marks Awarded:**

# ANSWERS

## 1. Type-1 vs Type-2 Hypervisors for a Mission-Critical Datacenter

A hypervisor is the layer that creates and runs virtual machines. Type-1 (bare-metal) hypervisors — ESXi, Hyper-V, KVM, Xen — install directly on the physical hardware. Type-2 (hosted) hypervisors — VirtualBox, VMware Workstation, Parallels — run as an application on top of a conventional host operating system (Windows, Linux, macOS).

### Comparison

| Factor | Type-1 (Bare-Metal) | Type-2 (Hosted) |
|---|---|---|
| **Performance** | Talks to hardware directly; near-native CPU, memory, and I/O throughput; minimal scheduling overhead. | Every instruction passes through the host OS first, adding a second layer of scheduling and I/O translation; noticeably slower under load. |
| **Security** | Small, purpose-built kernel (a thin attack surface); no general-purpose OS underneath to patch or exploit. | Inherits every vulnerability of the host OS; a compromised host OS can compromise every VM sitting on it. |
| **Manageability** | Centralized tools (vCenter, SCVMM, oVirt) for clustering, live migration, resource pools, and monitoring at scale. | Managed per-machine through the desktop application; no native clustering or centralized fleet management. |
| **Resource Isolation** | Strong — VMs are scheduled directly by the hypervisor, so one VM's load has limited effect on others. | Weaker — VMs compete with the host OS's own processes for the same underlying resources. |
| **Typical Use Case** | Datacenters, production servers, cloud infrastructure. | Developer laptops, quick testing, running a second OS alongside a primary desktop. |

### Justification

For a datacenter hosting mission-critical applications, a Type-1 hypervisor is the clear choice. Three reasons stand out:

- Performance headroom is non-negotiable for production workloads — the extra host-OS layer in Type-2 introduces latency that mission-critical systems (financial transactions, healthcare records, real-time services) can't absorb.
- Security posture matters more once real data and real users are on the line — a bare-metal hypervisor's minimal attack surface reduces the blast radius of any single vulnerability.
- Operating at scale requires centralized management — a datacenter with hundreds of VMs needs live migration, automated failover, and fleet-wide monitoring, none of which Type-2 hypervisors are built to provide.

Type-2 remains useful for development and testing, but it should not carry production or mission-critical load.

## 2. Isolating Finance, Healthcare, and Student Workloads on Shared Hardware

When a cloud provider hosts tenants with very different compliance and sensitivity profiles — finance (PCI-DSS), healthcare (HIPAA), and a student portal — on the same physical servers, isolation is enforced almost entirely through the virtualization layer, since the tenants share CPU, memory, storage, and network hardware underneath.

### How Virtualization Architecture Enforces Isolation

- **Compute isolation:** Each tenant's workload runs inside its own VM, scheduled and sandboxed by the hypervisor so that one VM cannot directly read another VM's memory or CPU state.
- **Network isolation:** Virtual switches and network overlays (VLANs, VXLAN, micro-segmentation) keep each tenant's traffic on a logically separate network even though it rides the same physical NICs and switches.
- **Storage isolation:** Virtual disks and storage volumes are logically partitioned per tenant, often with per-tenant encryption keys, so storage-layer access is scoped to the owning VM.
- **Resource isolation:** A hypervisor-enforced access control layer, combined with resource pools and quotas, stops one tenant's workload from starving another's (the "noisy neighbor" problem).
- **Policy isolation:** Separate identity domains, RBAC policies, and encryption key hierarchies per tenant ensure that even administrative access is scoped and audited per workload.

### Two Major Attack Surfaces

| Attack Surface | Description |
|---|---|
| **VM escape / hyperjacking** | A flaw in the hypervisor lets an attacker break out of their own VM and gain access to the hypervisor itself or to a neighboring tenant's VM — the single most severe risk in a shared-hardware model, since it defeats isolation entirely. |
| **Side-channel attacks via shared hardware** | Because VMs share physical CPU cores, caches, and memory buses, an attacker co-located on the same host can sometimes infer another tenant's data (e.g., encryption keys) by measuring timing or cache behavior, without ever breaking the VM boundary directly. |

Given these two risks, the provider should pair strong hypervisor patch management with workload placement policies (e.g., not co-locating a healthcare VM with an untrusted student-portal VM) and consider dedicated hosts for the most sensitive finance and healthcare tenants where compliance requires it.

## 3. VM Host Utilization Snapshot — Bottleneck Analysis

### Snapshot

| Metric | Observed Value | Healthy Range |
|---|---|---|
| **CPU utilization** | 92% | < 75–80% sustained |
| **Memory utilization** | 88% | < 80–85% sustained |
| **Storage I/O latency** | 35 ms | < 10–20 ms for typical workloads |
| **Average VM boot time** | 170 seconds | 30–60 seconds typical |

## Interpreting the Numbers

All four indicators are pointing the same direction: this host is resource-starved, and the symptoms are compounding each other rather than coming from one isolated cause.

- **CPU pressure:** CPU at 92% means VMs are regularly queued waiting for CPU time ("CPU ready" time climbs), which shows up as sluggish application response even when nothing has crashed.
- **Memory pressure:** Memory at 88% is close enough to the ceiling that the hypervisor is likely leaning on ballooning, compression, or swapping to host disk to keep VMs running — and swapping to disk is expensive.
- **Storage I/O latency:** 35 ms is high for typical enterprise storage (healthy SSD-backed storage usually sits under 10–20 ms); this points to an overloaded or under-provisioned storage backend, or too many VMs sharing the same datastore.
- **Boot time:** A 170-second boot is the clearest symptom of all three problems compounding — booting a VM is CPU-, memory-, and disk-I/O-intensive all at once, so when all three resources are already stretched thin, boot time balloons.

## Likely Root Cause

This is a classic case of VM sprawl / over-consolidation: too many VMs have been packed onto one host relative to its physical capacity. Memory pressure is forcing the hypervisor to swap, which drives up storage I/O latency, which in turn slows down every disk-dependent operation including VM boot — CPU contention on top of that means even the CPU-bound parts of boot-up are queued.

## Recommended Corrective Actions

- **Rebalance the load:** Migrate some VMs off this host to other hosts in the cluster (or trigger DRS/live-migration rebalancing) to immediately relieve CPU and memory pressure.
- **Increase memory headroom:** Add physical RAM to the host, or reduce over-provisioned VM memory allocations that aren't actually being used.
- **Address the storage bottleneck:** Move to faster storage (NVMe/SSD-backed datastores), spread VMs across more datastores, or check for a failing/saturated storage array.
- **Apply resource governance:** Set CPU and memory reservations/limits per VM so no single workload can monopolize the host, and identify any runaway processes.
- **Monitor proactively going forward:** Track CPU ready time, memory ballooning/swapping, and datastore latency over time to catch this kind of drift before it becomes user-visible.

Until the host is rebalanced, this is not a healthy host to place new production VMs on.

## 4. Software-Defined Datacenter (SDDC) vs Traditional Datacenter — Agility

A traditional datacenter ties compute, storage, and networking to specific physical hardware, so provisioning anything new usually means procuring, racking, cabling, and manually configuring equipment. A Software-Defined Datacenter abstracts all three layers into software, so infrastructure can be provisioned, reconfigured, and torn down through code rather than physical work — that abstraction is where the agility comes from.

### Compute Virtualization

In a traditional setup, an application is often tied to a specific physical server, so scaling means buying and installing new hardware. In an SDDC, compute is pooled — a hypervisor layer lets new virtual machines (or containers) be spun up on any available capacity in seconds, migrated between hosts without downtime, and auto-scaled in response to demand, all without touching physical hardware.

### Storage Virtualization

Traditional storage is usually allocated in fixed chunks per server or per SAN LUN, and resizing it is a manual, often disruptive operation. Software-Defined Storage pools disk capacity across the datacenter and exposes it as flexible, policy-driven volumes — capacity can be resized, replicated, tiered, or snapshotted on demand, and storage policies (replication factor, encryption, performance tier) can be attached per workload rather than per physical array.

### Network Virtualization

In a traditional datacenter, network changes (VLANs, firewall rules, routing) mean physically reconfiguring switches and routers, which is slow and error-prone. Network virtualization (software-defined networking, overlays like VXLAN, and network functions virtualization) lets the entire network topology — segments, firewalls, load balancers — be defined and changed in software, independent of the physical switching fabric underneath.

### Net Effect on Agility

| Dimension | Traditional Datacenter | SDDC |
|---|---|---|
| **Provisioning time** | Days to weeks (procurement, racking, manual setup) | Minutes (API/software-driven) |
| **Scaling** | Manual, hardware-bound | Automated, elastic, on-demand |
| **Change management** | Physical reconfiguration, higher risk of outages | Software-defined, version-controlled, repeatable |
| **Resource utilization** | Siloed per application, often over-provisioned | Pooled and shared, driven by policy |
| **Recovery from failure** | Manual failover, hardware-dependent | Automated, workload can move to healthy resources fast |

Because compute, storage, and networking are all defined in software, an SDDC can respond to a new requirement — a traffic spike, a new application, a compliance change — in minutes through automation, where a traditional datacenter would need physical intervention. That's the essence of the agility gain.

## 5. Multi-Cloud Federation Failure — Identity Mismatch

### The Problem

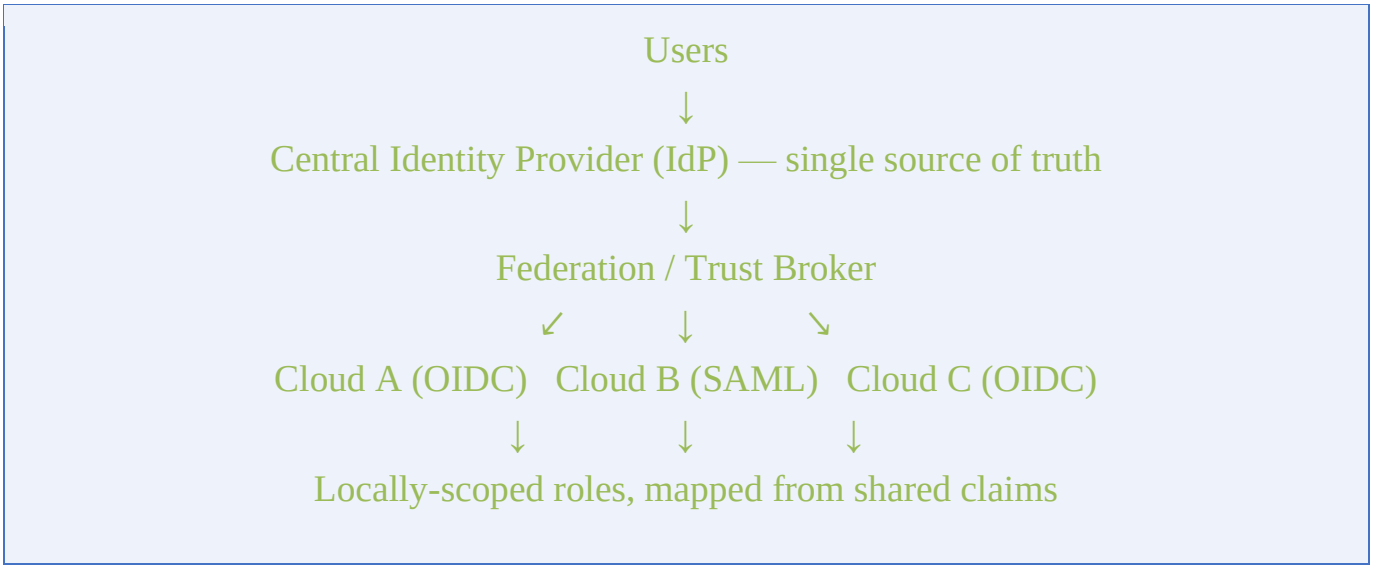
A multi-cloud federation lets a user authenticate once and access resources across several cloud providers without a separate login for each. When the project fails due to identity mismatches, it's almost always because each provider represents identity differently — different attribute names (e.g., one provider expects "email" as the unique identifier, another expects "UPN"), different token formats (SAML assertions vs. OAuth/OIDC tokens), inconsistent role or group naming, and no single, trusted source of truth for who a user is. The result: a user authenticated successfully at the identity provider but was

rejected, mis-mapped, or granted the wrong access once the request reached a different cloud.

### Root Causes to Analyze

- **Attribute schema mismatch:** Providers disagree on which attribute uniquely identifies a user, so the same person can register as two different identities in two clouds.
- **Protocol mismatch:** Mixing SAML-based providers with OAuth 2.0 / OpenID Connect-based providers without a translation layer between them.
- **Role/claim mapping gaps:** Role and group names aren't standardized, so a "Finance-Admin" role in one cloud doesn't map cleanly to an equivalent role in another.
- **Lack of synchronized provisioning:** No automated process to create, update, or revoke accounts across providers in sync, so an offboarded user can remain active in one cloud after being removed from another.
- **No single source of truth:** There isn't one authoritative identity provider that every cloud defers to, so trust relationships are built ad hoc and inconsistently.

### Proposed Federation Design




### Design Elements

- **Centralized identity provider:** Designate a single, authoritative Identity Provider that every participating cloud trusts, so there is exactly one place where a user's identity is created and managed.
- **Standardized federation protocol:** Use OpenID Connect / OAuth 2.0 for modern API-driven access and SAML 2.0 where legacy enterprise integration requires it, with

the identity provider issuing both from the same underlying identity so tokens stay consistent across protocols.

- **Canonical attribute mapping:** Define a common attribute schema (a fixed, agreed set of claims — unique ID, department, role, clearance level) that every provider maps into on its own side, so "who this user is" means the same thing everywhere.
- **Automated, synchronized provisioning:** Use SCIM (System for Cross-domain Identity Management) to automatically create, update, and deactivate accounts across all connected clouds the moment a change happens at the central IdP — closing the gap where a deprovisioned user stays active elsewhere.
- **Strong authentication and short-lived tokens:** Require MFA at the central IdP and issue short-lived, scoped tokens (rather than long-lived credentials) to each cloud, minimizing damage if a token is intercepted.
- **Privacy-preserving attribute release:** Encrypt identity attributes and tokens in transit and at rest, and only release the minimum attributes each cloud actually needs (data minimization) — especially relevant given that some attributes here relate to healthcare or financial roles.
- **Continuous auditing and reconciliation:** Log every authentication and authorization decision centrally, and periodically reconcile each cloud's local role assignments back against the central IdP to catch drift early.

### Why This Fixes the Failure

The original failure came from every cloud treating identity as something local and provider-specific. Centralizing identity at one trusted IdP, standardizing the protocol and attribute schema, and automating provisioning through SCIM removes the mismatches at their source — every cloud now consumes the same identity, expressed the same way, kept in sync automatically, rather than each maintaining its own inconsistent copy.